

RX2 Dashboard Truth Audit

Phase 1A source-of-truth inventory for Race Captain and Strategy metrics

Read-only audit

No code changes

Private dashboard only

Generated for race testing review

Executive Findings

Two active strategy chains:

Race Captain top cards use `generatePredictiveStrategy()`; native energy command cards use `buildRacePrediction()` -> `planBatterySwap()` -> `buildDeterministicStrategyRecommendation()`.

Most trust is Medium:

Most displayed strategy numbers are forecasts using live telemetry plus route/config fallbacks. Direct telemetry fields are High only when ESP32/cloud packets are fresh.

Stage 2 watch item:

Solar recovery and stopped/trailing recovery now exist in the prediction chain, but similar labels in older panels can use different energy-model assumptions.

Metric Inventory

METRIC	COMPONENT	DATA SOURCE	CALCULATION SOURCE	FILE	FUNCTION	SOURCE TYPE	TRUST
Recommended Speed	Race Captain top cards	Predictive strategy	<code>generatePredictiveStrategy -> recommendSpeed</code>	<code>components/DayCommandCenter.tsx; lib/strategyEngine.ts</code>	<code>recommendSpeed</code>	Prediction Engine	Medium
Recommended Speed	Native command card	Prediction + battery state	<code>buildRacePrediction -> planBatterySwap -> buildDeterministicStrategyRecommendation</code>	<code>components/DayCommandCenter.tsx; lib/deterministicStrategyRecommendation.ts</code>	<code>buildDeterministicStrategyRecommendation</code>	Prediction Engine	Medium
Mission Status	Race Captain / Strategy banner	Predictive strategy	<code>classifyMissionStatus(predictiveStrategy)</code>	<code>components/DayCommandCenter.tsx</code>	<code>classifyMissionStatus</code>	Derived Calculation	Medium
Race Health	Race Captain / Strategy banner	Predictive strategy + telemetry status	<code>calculateRaceHealth -> calculateRaceHealthBreakdown</code>	<code>components/DayCommandCenter.tsx</code>	<code>calculateRaceHealth</code>	Derived Calculation	Medium
Current SOC	Race Captain top cards	<code>telemetry.batterySocPercent</code>	Direct render	<code>components/DayCommandCenter.tsx</code>	<code>direct</code>	Live Telemetry	High if fresh
Alerts	Race Captain	Route, weather, telemetry, strategy	<code>raceCaptainAlerts</code> array	<code>components/DayCommandCenter.tsx</code>	<code>inline</code>	Derived Calculation	Medium
Next Critical Event	Race Captain	Route segments	<code>nextImportantSegment</code> lookup	<code>components/DayCommandCenter.tsx</code>	<code>inline route lookup</code>	Route Data	Medium
Strategy Recommendation	Race Captain native	Prediction, swap, battery state	<code>buildDeterministicStrategyRecommendation</code>	<code>lib/deterministicStrategyRecommendation.ts</code>	<code>buildDeterministicStrategyRecommendation</code>	Prediction Engine	Medium
Prediction Confidence	Prediction cards	Telemetry freshness, route, battery, schedule	<code>confidenceLevel</code>	<code>lib/racePrediction.ts</code>	<code>confidenceLevel</code>	Derived Calculation	Medium
Projected Next Stop SOC	Prediction summary / Next Event	Prediction engine + schedule	<code>projectedNextScheduleEventSocPercent ?? nextStop.socPercent</code>	<code>lib/racePrediction.ts; lib/raceSchedule.ts</code>	<code>buildRacePrediction</code>	Prediction Engine	Medium

Metric	Component	Data Source	Calculation Source	File	Function	Source Type	Trust
Projected End-Day SOC	Prediction summary / Strategy	Prediction engine + schedule	forecastRaceScheduleEnergy fallback projectEnergy	lib/racePrediction.ts; lib/raceSchedule.ts	forecastRaceScheduleEnergy	Prediction Engine	Medium
Predicted Wh/mi	Strategy prediction panel	Rolling history, live telemetry, config fallback	predictedWhPerMile; clamp 25-75	lib/racePrediction.ts	predictedWhPerMile	Derived Calculation	Medium
Predicted MPPT	Strategy prediction panel	MPPT telemetry then config fallback	predictedMpptWatts	lib/racePrediction.ts	predictedMpptWatts	Live Telemetry / Constant	Medium
Solar Recovery	Prediction summary	MPPT + schedule durations	forecastRaceScheduleEnergy	lib/raceSchedule.ts	forecastRaceScheduleEnergy	Prediction Engine	Medium
Stopped Recovery	Strategy prediction panel	MPPT + non-driving events	applyNonDrivingEvent	lib/raceSchedule.ts	applyNonDrivingEvent	Prediction Engine	Medium
Trailing Recovery	Strategy prediction panel	MPPT + trailer events	applyNonDrivingEvent	lib/raceSchedule.ts	applyNonDrivingEvent	Prediction Engine	Medium
Net Day Energy	Strategy prediction panel	Schedule forecast	drive Wh - recovered Wh	lib/raceSchedule.ts	forecastRaceScheduleEnergy	Derived Calculation	Medium
Swap Recommendation	Race Captain / Strategy	Battery state + prediction	planBatterySwap	lib/raceBatteryStrategy.ts	planBatterySwap	Swap Planner	Medium
Battery Strategy	Strategy summary	Race battery state	raceBatteryState + planBatterySwap	components/DayCommandCenter.tsx; lib/raceBatteryStrategy.ts	BatteryStrategySummaryPanel	Swap Planner / Manual Input	Medium
Trailing Recommendation	Race Captain / Strategy	Route lookahead + SOC + Wh/mi	analyzeRouteIntelligence -> estimateTrailingOption	lib/routeIntelligence.ts	estimateTrailingOption	Route Data / Prediction	Medium-Low
Current Wh/mi	Legacy Predictive Strategy panel	Telemetry or model fallback	getSafeWhPerMile inside generatePredictiveStrategy	components/PredictiveStrategyPanel.tsx; lib/strategyEngine.ts	getSafeWhPerMile	Derived Calculation	Medium
Model Wh/mi	Legacy Predictive Strategy panel	Energy simulation	simulateDayEnergy -> getSafeWhPerMile	lib/energy.ts; lib/strategyEngine.ts	simulateDayEnergy	Simulator	Low-Medium
Efficiency Delta	Legacy Predictive Strategy panel	Current vs model Wh/mi	(current - model) / model	lib/strategyEngine.ts	generatePredictiveStrategy	Derived Calculation	Medium
Thermal Risk	Legacy Predictive Strategy panel	Motor/controller telemetry	classifyThermalRisk	lib/strategyEngine.ts	classifyThermalRisk	Live Telemetry	High if fresh

Special Calculation Chains

Recommended Speed	Top cards: telemetry/route/car setup -> simulateDayEnergy() -> generatePredictiveStrategy() -> recommendSpeed(). Native cards: telemetry/history/route -> buildRacePrediction() -> planBatterySwap() -> buildDeterministicStrategyRecommendation().	Medium
Race Health	Telemetry source/status + predictive strategy -> calculateRaceHealthBreakdown() -> score and label.	Medium
Mission Status	Predictive strategy -> swap debug next-opportunity SOC + trailering action + highest route risk -> classifyMissionStatus().	Medium
Strategy Recommendation	Telemetry/history/route -> prediction -> swap planner -> battery state validation -> deterministic recommendation command, reason, speed, warnings.	Medium
Projected End-Day SOC	Telemetry battery energy or SOC estimate -> schedule events -> drive energy minus MPPT recovery -> projectedEndDaySocPercent.	Medium
Projected Next Stop SOC	Same prediction path, but projected to next schedule event or route stop rather than full day.	Medium
Predicted Wh/mi	Rolling recent Wh/mi -> live Wh/mi -> fallback 40 Wh/mi -> clamp 25-75 Wh/mi.	Medium
Solar Recovery	MPPT priority: charge power -> MPPT power -> solar power -> station fallback. Schedule applies recovery during driving, stops, and trailering.	Medium
Swap Recommendation	Active/spare pack SOC -> prediction SOC projections -> force swap 20%, plan swap 30%, spare advantage 10% -> action.	Medium
Trailering Recommendation	15-mile route lookahead + risk/type multipliers + current SOC + baseline Wh/mi -> projected driven/trailer SOC -> action.	Medium-Low

Legacy Assumption Audit

PATTERN	FILE / LINE	ACTIVE?	CLASSIFICATION
140	lib/deterministicStrategyRecommendation.test.ts:242	No runtime	Test SOC clamp fixture, not 140 Wh/mi.
140	lib/racePrediction.test.ts:186	No runtime	Test SOC clamp fixture, not 140 Wh/mi.
120	lib/raceCaptainEnergy.test.ts:108, 223	No runtime	Energy Wh fixture, not target Wh/mi.
120 cap	lib/telemetrySimulator.ts:117	Active	Current draw cap. Not legacy Wh/mi, but can create high simulated power.
24.8 mph	lib/esp32Telemetry.ts:67	Mock only	Mock packet speed, not strategy target.
24 Wh/mi	lib/telemetrySimulator.ts:147	Active	Descent efficiency. Above 20 floor, but optimistic.
Old thresholds 130/170/220	Repository search	Not found as active strategy bands	Remaining matches are route miles, gauge geometry, solar/current limits, or tests.
defaultRaceWhPerMile = 140	Repository search	Not found	Current config is 40.
defaultTargetSpeedMph = 24	Repository search	Not found	Current config is 35.

Suspicious or Inconsistent Metrics

- Race Captain top cards and native command cards can show different recommended speeds because they use different strategy chains.
- Race Health and Mission Status use the older predictive strategy swap advisor, while native Battery Strategy uses the newer race battery planner.
- There are two active swap planners: `batterySwapAdvisor.ts` and `raceBatteryStrategy.ts`.
- Solar recovery appears in several places but can be sourced from schedule forecast, integrated history, MPPT daily energy, or simulation depending on component.
- Simulator climb current can still reach 120A, which may produce harsher simulated power than the new Wh/mi targets suggest.
- Descent simulator efficiency is hardcoded at 24 Wh/mi, close to the lower realism floor.

Verify Before Race Testing

- Pick one authoritative strategy chain for Race Captain decisions.
- Confirm live ESP32/cloud telemetry includes reliable `batteryEnergyWh`; otherwise predictions estimate from SOC.
- Validate MPPT priority and units: `mpptChargePowerWatts`, `mpptPowerWatts`, `solarPowerWatts`.
- Compare Projected Next Stop SOC across prediction summary, next event, swap advisor, and debug panel.
- Review schedule stop/trailer durations because they directly drive solar recovery and end-day SOC.
- Recheck simulator current/power behavior after calibration, especially climb and descent cases.